

```

import tensorflow as tf
import tensorflow_datasets as tfds
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import time
from sklearn.metrics import confusion_matrix

# Disable eager execution for TensorFlow 1.x compatibility
tf.compat.v1.disable_eager_execution()

# Load MNIST dataset
mnist, info = tfds.load('mnist', with_info=True, as_supervised=True)

def preprocess(image, label, n_input=784, n_classes=10):
    image = tf.reshape(image, [n_input])
    image = tf.cast(image, tf.float32) / 255.0
    label = tf.one_hot(label, n_classes)
    return image, label

```

⚠ WARNING:tensorflow:From C:\Users\harsh\AppData\Local\Temp\ipykernel_18696\252612543.py:11: The name tf.disable_eager_execution is de

✓ Define Hyperparameter variations

```

# *****

# Define hyperparameter variations
# activation_functions = {'relu': tf.nn.relu, 'sigmoid': tf.nn.sigmoid, 'tanh': tf.nn.tanh} # Uncomment for all activation functions
activation_functions = {'relu': tf.nn.relu} # using only ReLU activation function

# hidden_layer_sizes_single_layer = [256, 128, 64] # Uncomment for single layer
hidden_layer_sizes_single_layer = [256]

hidden_layer_sizes_double_layer = [(160,100), (100,160), (100,100), (100,60), (60,60)] # Uncomment for double layer
# hidden_layer_sizes_double_layer = [(160,100)] # using only (160,100)

learning_rates = [1, 0.1, 0.01, 0.001] # Uncomment for all learning rates
# learning_rates = [1] # Using only 0.1 learning rate

# batch_sizes = [100, 10, 1] # Uncomment for all batch sizes
batch_sizes = [10] # Using only 10 batch size

# epochs_list = [100, 50, 10] # Uncomment for all epochs
epochs_list = [50] # Using only 50 epochs

# *****

```

```

print(
    '''
    Hyperparameter variations:
    Activation functions: {}
    Hidden layer sizes: {}
    Learning rates: {}
    Batch sizes: {}
    Epochs: {}
    '''.format(activation_functions.keys(), hidden_layer_sizes_double_layer, learning_rates, batch_sizes, epochs_list)
)

```



```

Hyperparameter variations:
Activation functions: dict_keys(['relu'])
Hidden layer sizes: [(160, 100), (100, 160), (100, 100), (100, 60), (60, 60)]
Learning rates: [1, 0.1, 0.01, 0.001]
Batch sizes: [10]
Epochs: [50]

```

```

# Create results directory
os.makedirs("results", exist_ok=True)

```

```
# Save metrics in txt
with open(f"results/HyperparameterVariations.txt", "w") as f:
    f.write('''Hyperparameter variations:
Activation functions: {}
Hidden layer sizes: {}
Learning rates: {}
Batch sizes: {}
Epochs: {}
'''.format(activation_functions.keys(), hidden_layer_sizes_double_layer, learning_rates, batch_sizes, epochs_list)
)
```

✓ This plots curves in epochs

```
...
Hyperparameter variations:
Activation functions: dict_keys(['relu'])
Hidden layer sizes: [(160, 100), (100, 160), (100, 100), (100, 60), (60, 60)]
Learning rates: [0.1,0.01,0.001]
Batch sizes: [10]
Epochs: [50]
...

# Loop through all combinations
for act_name, activation in activation_functions.items():
    for hidden_size in hidden_layer_sizes_double_layer:
        for lr in learning_rates:
            for batch_size in batch_sizes:
                for epochs in epochs_list:

                    # Prepare dataset
                    train_data = mnist['train'].map(preprocess).shuffle(60000).batch(
                        batch_size)
                    test_data = mnist['test'].map(preprocess).batch(batch_size)

                    # Define placeholders
                    X = tf.compat.v1.placeholder(tf.float32, [None, 784])
                    Y = tf.compat.v1.placeholder(tf.float32, [None, 10])

                    # Define weights and biases
                    weights = {
                        'h1': tf.Variable(tf.random.normal([784, hidden_size[0]])),
                        'h2': tf.Variable(tf.random.normal([hidden_size[0], hidden_size[1]])),
                        'out': tf.Variable(tf.random.normal([hidden_size[1], 10]))
                    }
                    biases = {
                        'b1': tf.Variable(tf.random.normal([hidden_size[0]])),
                        'b2': tf.Variable(tf.random.normal([hidden_size[1]])),
                        'out': tf.Variable(tf.random.normal([10]))
                    }

                    # Define neural network
                    def neural_net(x):
                        layer_1 = activation(tf.add(tf.matmul(x, weights['h1']), biases['b1']))
                        layer_2 = activation(tf.add(tf.matmul(layer_1, weights['h2']), biases['b2']))
                        out_layer = tf.matmul(layer_2, weights['out']) + biases['out']
                        return out_layer

                    logits = neural_net(X)
                    loss_op = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(
                        logits=logits, labels=Y))
                    optimizer = tf.compat.v1.train.AdamOptimizer(learning_rate=lr).minimize(loss_op)
                    correct_pred = tf.equal(tf.argmax(logits, 1), tf.argmax(Y, 1))
                    accuracy = tf.reduce_mean(tf.cast(correct_pred, tf.float32))

                    init = tf.compat.v1.global_variables_initializer()

                    # Lists for storing performance data
                    loss_history, accuracy_history, y_true, y_pred = [], [], [], []
                    start_time = time.time()

                    with tf.compat.v1.Session() as sess:
                        sess.run(init)
                        train_iterator = tf.compat.v1.data.make_initializable_iterator(
                            (train_data))
                        next_train_element = train_iterator.get_next()
                        sess.run(train_iterator.initializer)

                        for epoch in range(epochs):
                            epoch_loss, epoch_acc, batch_count = 0, 0, 0
```

```

while True:
    try:
        batch_x, batch_y = sess.run(next_train_element)
        _, loss, acc = sess.run([optimizer, loss_op,
                                accuracy], feed_dict={X: batch_x, Y: batch_y})
        epoch_loss += loss
        epoch_acc += acc
        batch_count += 1
    except tf.errors.OutOfRangeError:
        break
    epoch_loss /= batch_count
    epoch_acc /= batch_count
    loss_history.append(epoch_loss)
    accuracy_history.append(epoch_acc)
    sess.run(train_iterator.initializer)
    print(f"Epoch {epoch+1}/{epochs} - Loss: {epoch_loss:.4f},
    Accuracy: {epoch_acc:.3f}")

# Evaluate on test data
test_iterator = tf.compat.v1.data.make_initializable_iterator
(test_data)
next_test_element = test_iterator.get_next()
sess.run(test_iterator.initializer)
test_acc, test_count = 0, 0

while True:
    try:
        test_x, test_y = sess.run(next_test_element)
        acc, preds = sess.run([accuracy, tf.argmax(logits, 1)],
                                feed_dict={X: test_x, Y: test_y})
        y_true.extend(np.argmax(test_y, axis=1))
        y_pred.extend(preds)
        test_acc += acc
        test_count += 1
    except tf.errors.OutOfRangeError:
        break
test_acc /= test_count
execution_time = time.time() - start_time

# Save results
folder_name = f"results/{act_name}_H{hidden_size[0]}_{hidden_size
[1]}_LR{lr}_B{batch_size}_E{epochs}"
os.makedirs(folder_name, exist_ok=True)

# Plot loss curve
plt.figure()
plt.plot(range(epochs), loss_history, label='Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Loss Curve')
plt.legend()
plt.savefig(f"{folder_name}/loss_curve.png")
plt.close()

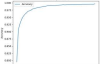
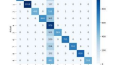
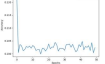
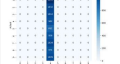
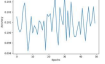
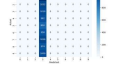
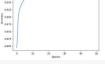
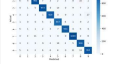
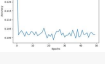
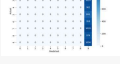
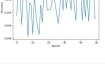
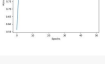
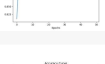
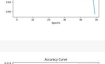
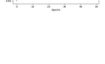
# Plot accuracy curve
plt.figure()
plt.plot(range(epochs), accuracy_history, label='Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve')
plt.legend()
plt.savefig(f"{folder_name}/accuracy_curve.png")
plt.close()

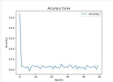
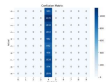
# Plot and save confusion matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=range(10), yticklabels=range(10))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.savefig(f"{folder_name}/confusion_matrix.png")
plt.close()

# Save metrics in csv
with open(f"{folder_name}/metrics.csv", "w") as f:
    f.write(f"ActivationFunction,HiddenSize,LearningRate,
    BatchSize,NumberOfEpochs,Test Accuracy,Execution Time(in sec)/
    n{act_name},{hidden_size[0]} and {hidden_size[1]}},{lr},
    {batch_size},{epochs},{test_acc},{execution_time}")

```

Results of all the Variations

ActivationFunction	HiddenSize	LearningRate	BatchSize	NumberOfEpochs	TestAccuracy	ExecutionTimeInSeconds	LossCurve	AccuracyCurve	ConfusionMatrix
relu	(100 and 100)	0.001	10	50	0.963702380657196	1221.1444935798645			
relu	(100 and 100)	0.01	10	50	0.7210999727249146	657.3895020484924			
relu	(100 and 100)	0.1	10	50	0.09819955378770828	603.5365436077118			
relu	(100 and 100)	1	10	50	0.10099950432777405	458.0438141822815			
relu	(100 and 160)	0.001	10	50	0.9660019278526306	1157.8503274917603			
relu	(100 and 160)	0.01	10	50	0.8288005590438843	487.95741868019104			
relu	(100 and 160)	0.1	10	50	0.10089950263500214	419.38601183891296			
relu	(100 and 160)	1	10	50	0.09739954024553299	406.1002399921417			
relu	(100 and 60)	0.001	10	50	0.9645021557807922	954.3139564990997			
relu	(100 and 60)	0.01	10	50	0.9279025197029114	647.4823021888733			
relu	(100 and 60)	0.1	10	50	0.1134994700551033	612.3821523189545			
relu	(100 and 60)	1	10	50	0.10089950263500214	500.68239283561707			
relu	(160 and 100)	0.001	10	50	0.9704017043113708	1067.8954167366028			
relu	(160 and 100)	0.01	10	50	0.6314007043838501	489.5189564228058			
relu	(160 and 100)	0.1	10	50	0.1134994700551033	384.7974236011505			
relu	(160 and 100)	1	10	50	0.08919956535100937	361.02485752105713			
relu	(60 and 60)	0.001	10	50	0.9583021998405457	1105.6882936954498			
relu	(60 and 60)	0.01	10	50	0.8588016033172607	877.531261920929			

ActivationFunction	HiddenSize	LearningRate	BatchSize	NumberOfEpochs	TestAccuracy	ExecutionTimeInSeconds	LossCurve	AccuracyCurve	ConfusionMatrix
relu	(60 and 60)	0.1	10	50	0.09819955378770828	668.0333743095398			
relu	(60 and 60)	1	10	50	0.10279951989650726	560.3649253845215		